

RCBF_SAC 5.2 初学者逐行讲解

文档目录

文档目录	2
目录和阅读方法	5
1. 先建立 Python 到 C/C++ 的翻译表	5
1.1 Python 为什么看不见参数类型	5
1.2 点号 . 到底是什么	6
1.3 圆括号 () 的三种常见意义	6
1.4 方括号 [] 的常见意义	6
1.5 shape 是什么	7
1.6 本文涉及的主要类型	7
1.7 NumPy 数组和 Tensor 的区别	7
1.8 初学者应该怎样读一行链式调用	8
2. __init__：创建 Agent	8
2.1 函数声明和参数类型	8
2.2 保存 SAC 标量参数	9
2.3 保存其余配置	9
2.4 选择 CPU 或 GPU	9
2.5 创建普通 Critic	10
2.6 创建 Critic 优化器	10
2.7 创建 Target Critic 并复制参数	11
2.8 保存 CBF 模式	11
2.9 Gaussian Policy 分支	11
计算 target entropy	11
创建可训练的 log_alpha	12
创建 Policy	12
2.10 Deterministic Policy 分支	13
2.11 创建 CBF-QP 安全层	13
2.12 创建可选 Compensator	13
2.13 构造完成后的成员类型	14
3. select_action：从 observation 得到环境动作	14
3.1 参数和返回值	14
3.2 把输入转换成 Tensor	15
3.3 可选转换 cbf_info	15
3.4 判断单样本还是 batch	16

3.5 为单样本增加 batch 维	16
3.6 Warmup 随机动作	16
3.7 非 Warmup 时使用 Policy	17
3.8 加上可选 Compensator 动作	18
3.9 是否经过安全层	18
3.10 返回 NumPy 数组	19
3.11 select_action 的完整数据流	19
4. update_parameters：一次 SAC 网络更新	19
4.1 参数类型	20
4.2 经验中保存了什么	20
4.3 判断是否混合模型经验	20
4.4 从真实经验池采样	21
4.5 从模型经验池采样	21
4.6 拼接真实和模型数据	21
4.7 Model-free 分支	22
4.8 NumPy 转换成 Tensor	22
4.9 with torch.no_grad() 是什么	23
4.10 为下一个状态生成动作	23
4.11 full 和 mod 的安全动作	23
4.12 Target Critic 计算下一个状态价值	23
4.13 Bellman 目标	24
4.14 普通 Critic 当前预测	24
4.15 两个 Critic 的 MSE	24
4.16 更新 Critic 的固定三步	25
4.17 为 Policy 更新重新采样动作	25
4.18 Policy 更新是否经过可微安全层	25
4.19 Critic 评价当前 Policy 动作	25
4.20 Policy Loss	26
4.21 自动更新 Alpha	26
4.22 Target Critic 软更新	27
4.23 返回日志数字	27
5. update_parameters_compensator	27
6. save_model：保存网络参数	28
6.1 参数	28
6.2 打印保存位置	28

6.3 保存 Actor	28
6.4 保存 Critic	29
6.5 可选保存 Compensator	29
6.6 当前没有保存的内容	29
7. load_weights：加载网络参数	29
7.1 参数和提前返回	29
7.2 加载 Actor	29
7.3 加载 Critic	30
7.4 加载可选 Compensator	30
7.5 一个重要的续训限制	30
8. get_safe_action：调用动力学模型和 CBF-QP	30
8.1 参数和返回类型	31
8.2 Observation 转物理 State	31
8.3 预测扰动	31
8.4 调用 CBF-QP 层	32
8.5 返回	32
9. 七个函数怎样连起来	32
9.1 程序启动	32
9.2 与真实环境交互	32
9.3 网络更新	33
9.4 保存和测试	33
10. 四种 CBF 模式在这些函数中的位置	33
off	33
baseline	33
full	33
mod	33
11. 看到这些写法时应该立刻翻译	33
12. 最后应该掌握的对象关系	34

适用对象：学过 C/C++，但几乎没有 Python、NumPy、PyTorch 和强化学习经验的读者。

讲解对象：[打开 `rcbf\\_sac/sac\\_cbf.py`](#)

本文对应 [打开 `TODO.md` 第 5.2 节](#)。

本文以当前工作区中的代码和行号为准。代码继续修改以后，行号可能发生变化，但函数名不会因此改变。

目录和阅读方法

不要试图第一次就记住 SAC 的所有公式。先完成三个较小目标：

1. 看见一个名字时，能判断它大概是普通数字、数组、Tensor，还是某个类的对象。
2. 看见点号、圆括号和方括号时，能判断程序正在访问成员、调用函数，还是读取数组元素。
3. 看见一行 PyTorch 代码时，先追踪数据的类型和 `shape`，再考虑它的数学含义。

本文讲解 `RCBF_SAC` 的全部七个函数：

- `__init__`
- `select_action`
- `update_parameters`
- `update_parameters_compensator`
- `save_model`
- `load_weights`
- `get_safe_action`

1. 先建立 Python 到 C/C++ 的翻译表

1.1 Python 为什么看不见参数类型

Python 函数可以写成：

```
def select_action(self, state, dynamics_model, evaluate=False):
```

它没有像 C++ 一样把类型写在参数前面。接近 C++ 的伪声明是：

```
SelectActionResult select_action(  
    RCBF_SAC* this_,  
    NumpyArrayOrTensor state,  
    DynamicsModel& dynamics_model,  
    bool evaluate = false  
);
```

Python 采用动态类型。变量名本身没有永久固定的类型，运行到赋值语句时，它才引用右侧的对象。

```
x = 3  
x = "hello"
```

第一行之后，`x` 指向整数对象。第二行之后，`x` 改为指向字符串对象。科研代码通常不会故意这样乱改类型，但 Python 语法允许这样做。

阅读本项目时，要从三个地方推断类型：

- 这个参数从哪里传进来。
- 右侧调用了哪个构造函数。
- 后续对它使用了哪些成员和运算。

1.2 点号 . 到底是什么

点号表示“进入这个对象，访问它里面的名字”。

```
obj.value  
obj.run()
```

近似 C++：

```
obj.value;  
obj.run();
```

关键区别看圆括号：

- **obj.value**：读取成员属性，当前没有调用函数。
- **obj.run()**：调用成员函数。
- **obj.run**：只取得函数对象本身，没有执行它。

本文件中的常见例子：

```
action_space.shape  
self.policy.parameters()  
tensor.to(self.device)
```

- **.shape** 是属性。
- **.parameters()** 是函数调用。
- **.to(...)** 也是函数调用。

1.3 圆括号 () 的三种常见意义

第一种，调用函数：

```
hard_update(target, source)
```

第二种，创建对象，本质上仍会触发函数调用：

```
QNetwork(7, 2, 256)  
Adam(parameters, lr=0.0003)
```

第三种，数学分组：

```
(1 - real_ratio) * batch_size
```

1.4 方括号 [] 的常见意义

读取第几个元素：

```
shape[0]
```

创建列表：

```
[self.log_alpha]
```

向数组某个位置写入：

```
action[i] = value
```

1.5 shape 是什么

shape 描述数组每个方向有多少个元素。

```
shape = (7,)  一个长度为 7 的一维数组  
shape = (1, 7) 1 行 7 列  
shape = (256, 7) 256 行, 每行 7 个数字
```

(7,) 是 Python 元组 **tuple**。只有一个元素的元组必须保留逗号, 否则 **(7)** 只是普通整数 7 外面加了括号。

1.6 本文涉及的主要类型

- **int** : 整数, 例如 **256**。
- **float** : 浮点数, 例如 **0.99**。
- **bool** : **True** 或 **False**。
- **str** : 字符串, 例如 **"Gaussian"**。
- **NoneType** : **None**, 表示当前没有有效对象。
- **tuple** : 长度固定的一组对象, 例如 **(2,)**。
- **list** : 可变长度列表, 例如 **[tensor]**。
- **numpy.ndarray** : NumPy 数组, 通常位于 CPU 普通内存。
- **torch.Tensor** : PyTorch 多维数组, 可以在 CPU 或 GPU 上, 还可以记录梯度。
- **gym.spaces.Box** : 连续动作空间对象, 保存动作上下界和维数。
- **argparse.Namespace** : 命令行参数对象。
- **QNetwork**、**GaussianPolicy** : 本项目定义的神经网络对象。
- **Adam** : PyTorch 优化器对象。
- **ReplayMemory** : 本项目定义的经验回放池对象。
- **DynamicsModel** : 本项目定义的动力学和 GP 扰动模型对象。

1.7 NumPy 数组和 Tensor 的区别

两者都可以理解成“带形状的多维数字数组”, 但用途不同。

```
numpy.ndarray 常用于环境、数据整理和普通数值计算  
torch.Tensor 常用于神经网络、GPU 计算和自动求梯度
```

常见转换路线：

```
NumPy ndarray  
-> torch.FloatTensor(...)  
-> .to(device)  
-> Tensor
```

从 Tensor 返回环境时：

```
Tensor
-> .detach()
-> .cpu()
-> .numpy()
-> NumPy ndarray
```

1.8 初学者应该怎样读一行链式调用

代码：

```
final_action.detach().cpu().numpy()[0]
```

永远从左向右拆开：

```
tmp1 = final_action.detach()
tmp2 = tmp1.cpu()
tmp3 = tmp2.numpy()
result = tmp3[0]
```

链式写法只是把中间变量省略了，没有产生新的语法规则。

2. `__init__`：创建 Agent

源码入口：[跳到 `\_\_init\_\_`](#)

2.1 函数声明和参数类型

```
def __init__(self, num_inputs, action_space, env, args):
```

接近 C++：

```
RCBF_SAC(
    int num_inputs,
    gym::Box& action_space,
    GymEnvironment& env,
    argparse::Namespace& args
);
```

参数逐个解释：

- `self`：RCBF_SAC 对象本身，接近 C++ 的 `this`。
- `num_inputs`：int，策略网络输入维数。Unicycle 中是 7。
- `action_space`：gym.spaces.Box 对象，保存动作维数、上下界和随机采样函数。
- `env`：环境对象。实际类型可能是 `UnicycleEnv`、`SimulatedCarsEnv` 或 `PvtolEnv`。
- `args`：argparse.Namespace 对象，里面保存命令行参数。

创建位置：[跳到 `main.py` 创建 agent 的代码](#)

```
agent = RCBF_SAC(
    env.observation_space.shape[0],
    env.action_space,
    env,
    args
)
```

以 Unicycle 为例，实际含义接近：


```
agent = RCBF_SAC(7, 一个Box对象, 一个UnicycleEnv对象, 一个Namespace对象)
```

2.2 保存 SAC 标量参数

```
self.gamma = args.gamma
self.tau = args.tau
self.alpha = args.alpha
```

三行右侧都是 **float**。

默认内容：

```
gamma = 0.99
tau = 0.005
alpha = 0.2
```

近似 C++：

```
this->gamma = args.gamma;
this->tau = args.tau;
this->alpha = args.alpha;
```

Python 不要求先在类声明中列出成员。第一次执行 **self.gamma = ...** 时，当前对象才获得 **gamma** 属性。

2.3 保存其余配置

```
self.policy_type = args.policy
```

- 类型：**str**
- 默认内容：**"Gaussian"**

```
self.target_update_interval = args.target_update_interval
```

- 类型：**int**
- 默认内容：**1**

```
self.automatic_entropy_tuning = args.automatic_entropy_tuning
```

- 类型：**bool**
- 默认内容：**True**

```
self.action_space = action_space
```

- 类型：**gym.spaces.Box**
- 含义：保存传进来的动作空间对象。

这里通常没有复制完整对象。可以先近似理解成保存一个指向同一对象的引用。

2.4 选择 CPU 或 GPU

```
self.device = torch.device("cuda" if args.cuda else "cpu")
```

A if condition else B 是 Python 条件表达式，接近 C++：

```
condition ? A : B
```

因此：

```
args.cuda 为 True -> torch.device("cuda")
```

args.cuda 为 False -> torch.device("cpu")

`torch.device(...)` 创建的是 PyTorch 设备对象，不是普通字符串。

2.5 创建普通 Critic

```
self.critic = QNetwork(
    num_inputs,
    action_space.shape[0],
    args.hidden_size
).to(device=self.device)
```

从内向外分析。

第一步：

```
action_space.shape
```

Unicycle 动作是 `[v, omega]`，所以内容是：

```
(2,)
```

类型是 `tuple`。

第二步：

```
action_space.shape[0]
```

读取元组第 0 个元素，得到 `int` 类型的 2。

第三步：

```
QNetwork(num_inputs, 2, args.hidden_size)
```

默认 Unicycle 中近似：

```
QNetwork(7, 2, 256)
```

创建一个 `QNetwork` 对象。这个对象内部同时包含 Q1 和 Q2 两套网络。

`QNetwork` 定义：[跳到 `model.py` 的 `QNetwork`](#)

第四步：

```
.to(device=self.device)
```

这是成员函数调用。它把神经网络中的参数移动到 `self.device` 指定的 CPU 或 GPU。

最终：

```
self.critic 的类型 = QNetwork
```

2.6 创建 Critic 优化器

```
self.critic_optim = Adam(self.critic.parameters(), lr=args.lr)
```

先执行：

```
self.critic.parameters()
```

`.parameters()` 是函数调用，返回 critic 内所有需要学习的 Tensor 参数。

然后：

```
Adam(..., lr=args.lr)
```

创建 `torch.optim.Adam` 对象。 `lr=` 是关键字参数，表示把右侧值传给名称为 `lr` 的参数。

默认学习率：

```
args.lr = 0.0003
```

最终：

```
self.critic_optim 的类型 = Adam
```

优化器不等于神经网络。神经网络负责计算，优化器负责依据梯度修改网络参数。

2.7 创建 Target Critic 并复制参数

```
self.critic_target = QNetwork(...).to(self.device)
```

这又创建了一个独立的 `QNetwork` 对象。

此时有两套网络：

```
self.critic    直接通过梯度训练
self.critic_target 缓慢跟随，用来计算稳定目标
```

```
hard_update(self.critic_target, self.critic)
```

这是普通函数调用，不是成员函数调用。函数来自：

[跳到 `utils.py` 的 `hard\\_update`](#)

参数方向是：

```
第一个参数 target：接收数据
第二个参数 source：提供数据
```

执行后：

```
critic_target 的参数 = critic 的参数
```

2.8 保存 CBF 模式

```
self.cbf_mode = args.cbf_mode
```

- 类型：`str`
- 可能内容：`"off"`、`"baseline"`、`"full"`、`"mod"`

2.9 Gaussian Policy 分支

```
if self.policy_type == "Gaussian":
```

`==` 比较两个值是否相等。默认配置会进入这个分支。

```
if self.automatic_entropy_tuning is True:
```

`is True` 检查对象是否就是布尔值 `True`。在这里可以先理解成“如果启用了自动熵调节”。

计算 target entropy

```
self.target_entropy = -torch.prod(
    torch.Tensor(action_space.shape).to(self.device)
).item()
```

Unicycle 中逐步变化：

```
action_space.shape
    类型 tuple, 内容 (2,)

torch.Tensor(action_space.shape)
    类型 Tensor, 内容 tensor([2.]), shape (1,)

.to(self.device)
    类型仍是 Tensor, 只改变所在设备

torch.prod(...)
    把所有元素相乘, 得到只含一个数的 Tensor : tensor(2.)

前面的负号 -
    得到 tensor(-2.)

.item()
    从单元素 Tensor 取出普通 Python 数字 -2.0
```

所以：

```
self.target_entropy 的类型 = float
self.target_entropy 的值 = -2.0
```

创建可训练的 log_alpha

```
self.log_alpha = torch.zeros(
    1,
    requires_grad=True,
    device=self.device
)
```

结果：

```
类型：torch.Tensor
内容：tensor([0.])
shape：(1,)
设备：CPU 或 GPU
requires_grad：True
```

requires_grad=True 表示 PyTorch 要记录与它相关的运算，以便以后执行 **backward()** 求梯度。

```
self.alpha_optim = Adam([self.log_alpha], lr=args.lr)
```

[self.log_alpha] 创建一个 Python **list**，里面只有一个 Tensor。Adam 接收“一组待优化参数”，所以即使只有一个参数，也把它放入列表。

创建 Policy

```
self.policy = GaussianPolicy(
    num_inputs,
    action_space.shape[0],
    args.hidden_size,
    action_space
).to(self.device)
```

最终类型：

```
self.policy = GaussianPolicy 对象
```

Policy 定义：[跳到 'GaussianPolicy'](#)

然后创建它自己的优化器：

```
self.policy_optim = Adam(self.policy.parameters(), lr=args.lr)
```

critic 和 policy 使用不同优化器，彼此不会自动混用参数。

2.10 Deterministic Policy 分支

```
else:
    self.alpha = 0
    self.automatic_entropy_tuning = False
    self.policy = DeterministicPolicy(...)
    self.policy_optim = Adam(...)
```

只有 `policy_type` 不是 "Gaussian" 时才进入。

- `self.alpha`：设置为整数 0，但参与 Tensor 运算时可自动转换。
- 自动熵调节关闭。
- `self.policy` 类型改为 `DeterministicPolicy`。

本项目默认训练一般不进入这个分支。

2.11 创建 CBF-QP 安全层

```
self.env = env
```

保存环境对象引用。

```
self.cbf_layer = None
```

None 表示当前没有有效的安全层对象，可以近似联想到 C++ 的 `nullptr`，但两者不是完全相同的语言机制。

```
if self.cbf_mode != 'off':
```

!= 表示不等于。只要模式不是 "off"，就执行：

```
self.cbf_layer = CBFQPLayer(
    env,
    args,
    args.gamma_b,
    args.k_d,
    args.l_p
)
```

执行后：

```
self.cbf_layer 的类型 = CBFQPLayer
```

定义位置：[跳到 `CBFQPLayer`](#)

2.12 创建可选 Compensator

```
if args.use_comp:
```

如果 `args.use_comp` 为真：

```
self.compensator = Compensator(
    num_inputs,
    action_space.shape[0],
    action_space.low,
    action_space.high,
    args
)
```

参数类型和内容：

- `num_inputs` : `int`。
- `action_space.shape[0]` : `int`，动作维数。
- `action_space.low` : `numpy.ndarray`，动作下界。
- `action_space.high` : `numpy.ndarray`，动作上界。
- `args` : `argparse.Namespace`。

Unicycle 中：

```
action_space.low = [-1.0, -1.0]
action_space.high = [ 1.0, 1.0]
```

否则：

```
self.compensator = None
```

Compensator 定义：[跳到`compensator.py`](#)

2.13 构造完成后的成员类型

默认 Unicycle、Gaussian policy、`cbf_mode="full"` 下：

```
agent.gamma      float
agent.tau        float
agent.alpha      float, 训练后可能变成单元素 Tensor
agent.policy_type str
agent.device     torch.device
agent.action_space gym.spaces.Box
agent.env        UnicycleEnv
agent.critic     QNetwork
agent.critic_target QNetwork
agent.critic_optim Adam
agent.policy     GaussianPolicy
agent.policy_optim Adam
agent.log_alpha  Tensor, shape (1,)
agent.alpha_optim Adam
agent.cbf_layer  CBFQPLayer
agent.compensator None
```

3. `select_action`：从 observation 得到环境动作

源码入口：[跳到`select\\_action`](#)

3.1 参数和返回值

```
def select_action(
    self,
    state,
    dynamics_model,
    evaluate=False,
    warmup=False,
    safe_action=True,
    cbf_info=None
):
```

参数：

- **self** : RCBF_SAC 对象。
- **state** : 通常是 `numpy.ndarray`, 也允许 `torch.Tensor`。名字叫 state, 但主循环传入的实际是 observation。
- **dynamics_model** : `DynamicsModel` 对象。
- **evaluate** : `bool`。False 时随机采样, True 时取均值动作。
- **warmup** : `bool`。True 时不用 policy, 直接随机动作。
- **safe_action** : `bool`。True 时经过 CBF-QP。
- **cbf_info** : 通常是 None 或 `numpy.ndarray`, 主要给 PVTOL 额外安全约束使用。

默认值写在 = 后面。调用者不传该参数时, Python 自动使用默认值。

主循环调用：[跳到 'main.py' 的动作选择](#)

没有 compensator 时返回两个 NumPy 数组：

```
final_action, cbf_action
```

启用 compensator 时返回三个：

```
final_action, action_comp, cbf_action
```

这叫动态返回结构。Python 允许它, 但确实比 C++ 的固定返回类型更难读。

3.2 把输入转换成 Tensor

```
state = to_tensor(state, torch.FloatTensor, self.device)
```

to_tensor 是普通函数：

[跳到 'utils.py' 的 'to_tensor'](#)

传入：

- **state** : NumPy 数组或已经存在的 Tensor。
- **torch.FloatTensor** : 希望使用 32 位浮点 Tensor。
- **self.device** : CPU 或 GPU。

如果输入是 NumPy 数组, 返回 `torch.Tensor`。如果已经不是 NumPy 数组, 当前实现直接原样返回。

Unicycle 单个 observation 的典型变化：

```
转换前：numpy.ndarray, shape (7,)
转换后：torch.Tensor, shape (7,)
```

3.3 可选转换 cbf_info

```
if cbf_info:
    cbf_info = to_tensor(cbf_info, torch.FloatTensor, self.device)
```

if cbf_info: 使用 Python 的隐式真假判断。

- **None** 会被当作假。
- 空容器通常会被当作假。

- 非空对象通常会被当作真。

实现注意：NumPy 多元素数组和 Tensor 不适合直接写进 `if`，可能出现“真假不明确”的异常。更稳定的写法通常是 `if cbf_info is not None:`。这里讲的是当前代码行为，不在本节修改它。

3.4 判断单样本还是 batch

```
expand_dim = len(state.shape) == 1
```

逐步拆解：

```
state.shape    tuple, 例如 (7,) 或 (256, 7)
len(state.shape) int, shape 中有几个数字
== 1          比较, 返回 bool
```

例子：

```
单样本 shape (7,)   -> len 为 1 -> expand_dim=True
批量 shape (256, 7) -> len 为 2 -> expand_dim=False
```

变量名 `expand_dim` 表示“是否需要增加 batch 维”。

3.5 为单样本增加 batch 维

```
if expand_dim:
    state = state.unsqueeze(0)
```

`.unsqueeze(0)` 是 Tensor 成员函数，在第 0 个位置增加一个长度为 1 的维度。

```
调用前 shape (7,)
调用后 shape (1, 7)
```

神经网络通常统一接收：

```
(batch_size, feature_count)
```

即使只有一个样本，也写成 `(1, 7)`。

```
if cbf_info:
    cbf_info = cbf_info.unsqueeze(0)
```

如果有额外 CBF 信息，也增加相同的 batch 维。

3.6 Warmup 随机动作

```
if warmup:
```

训练前 `start_steps` 步，主循环传入 `warmup=True`。

```
batch_size = state.shape[0]
```

`state` 现在一定带 batch 维。

```
state.shape = (1, 7) -> batch_size = 1
state.shape = (20, 7) -> batch_size = 20
```

```
action = torch.zeros(
    (batch_size, self.action_space.shape[0])
).to(self.device)
```

先创建全零 Tensor。

Unicycle 单样本：


```
batch_size = 1
action_dim = 2
action.shape = (1, 2)
action 内容初始为 [[0.0, 0.0]]
```

.to(self.device) 把 Tensor 移动到 CPU 或 GPU。

```
for i in range(batch_size):
```

接近 C++：

```
for (int i = 0; i < batch_size; ++i)
```

range(batch_size) 产生从 0 到 **batch_size - 1** 的整数。

```
self.action_space.sample()
```

调用 Gym Box 的随机采样函数，返回 **numpy.ndarray**。

Unicycle 中可能得到：

```
[0.37, -0.81], shape (2,)
```

```
torch.from_numpy(...)
```

把 NumPy 数组转换成 Tensor。

```
action[i] = ...
```

把随机动作写入第 **i** 行。

3.7 非 Warmup 时使用 Policy

```
else:
    if evaluate is False:
        action, _, _ = self.policy.sample(state)
    else:
        _, _, action = self.policy.sample(state)
```

self.policy.sample(state) 返回一个三元素 tuple：

(随机动作, 动作的 log 概率, 均值动作)

Policy 实现：[跳到 'GaussianPolicy.sample'](#)

Python 支持 tuple 解包：

```
a, b, c = function()
```

等价思路：

```
tmp = function()
a = tmp[0]
b = tmp[1]
c = tmp[2]
```

单独的 **_** 是一个合法变量名。程序员用它表示“这个返回值我不关心”。

训练模式：

```
action, _, _ = self.policy.sample(state)
```

取得第一个返回值，也就是带随机性的采样动作。

评估模式：

```
_, _, action = self.policy.sample(state)
```

取得第三个返回值，也就是均值动作。

两种情况下，Unicycle 单样本的 **action** 都是 Tensor，shape 为 (1, 2)。

注意执行顺序：**warmup=True** 时程序不会进入 policy 分支。因此 warmup 的优先级高于 evaluate。

3.8 加上可选 Compensator 动作

```
if self.compensator:
```

如果成员不是 **None**，对象通常被视为真。

```
    action_comp = self.compensator(state)
```

这里看起来像函数调用，但 **self.compensator** 是对象。Python 对象可以实现 `__call__`，从而允许写成 **对象(...)**。

Compensator 的调用实现：[跳到 `Compensator.\\_\\_call\\_\\_`](#)

返回：

```
    action_comp : torch.Tensor
    shape 与 action 相同，例如 (1, 2)

    action += action_comp
```

+= 是原地加法的写法。概念上接近：

```
    action = action + action_comp
```

此后 **action** 表示：

policy 动作 + compensator 动作

3.9 是否经过安全层

```
if safe_action:
    final_action = self.get_safe_action(
        state,
        action,
        dynamics_model,
        cbf_info_batch=cbf_info
    )
```

这是调用当前对象自己的成员函数。**cbf_info_batch=...** 是关键字传参。

返回：

```
    final_action : torch.Tensor
    shape 与 action 相同
    内容是经过 CBF-QP 后的最终安全动作

    cbf_action = final_action - action
```

根据定义：

final_action = 原动作 + CBF修正

所以：

```
    cbf_action = CBF修正
```

如果 **safe_action=False**：

```
    final_action = action
    cbf_action = torch.zeros_like(final_action)
```

`.zeros_like(x)` 创建与 `x` 类型、shape、设备相同的全零 Tensor。

3.10 返回 NumPy 数组

神经网络内部使用 Tensor，但 Gym 环境通常接收 NumPy 数组，因此返回前要转换。

```
if not self.compensator:
```

not 是逻辑非。如果 `compensator` 为 **None**，条件为真。

单样本分支：

```
return (
    final_action.detach().cpu().numpy()[0],
    cbf_action.detach().cpu().numpy()[0]
)
```

逐步分析：

```
.detach() 得到不再连接自动求导图的 Tensor
.cpu()   把 Tensor 移到 CPU
.numpy() 转成 numpy.ndarray
[0]      取 batch 中第 0 个样本
```

shape 变化：

```
Tensor (1, 2)
-> ndarray (1, 2)
-> [0]
-> ndarray (2,)
```

批量输入时不执行 `[0]`，返回 shape `(B, action_dim)`。

启用 `compensator` 时，函数返回三个数组：

```
return final_action, action_comp, cbf_action
```

3.11 `select_action` 的完整数据流

```
NumPy observation
-> Tensor
-> 增加 batch 维
-> 随机动作或 policy 动作
-> 可选 compensator
-> 可选 CBF-QP
-> detach
-> CPU
-> NumPy
-> 去掉单样本 batch 维
-> 返回给环境
```

4. `update_parameters`：一次 SAC 网络更新

源码入口：[跳到 `update\_parameters`](#)

这是本文件最长、最重要，也最容易痛苦的函数。不要把它当成一个整体。它实际由六段组成：

1. 从 replay buffer 采样。
2. NumPy 转 Tensor。
3. 计算 critic 的学习目标。

4. 更新 critic。
5. 更新 policy 和 alpha。
6. 软更新 target critic 并返回日志数字。

4.1 参数类型

```
def update_parameters(
    self,
    memory,
    batch_size,
    updates,
    dynamics_model,
    memory_model=None,
    real_ratio=None
):
```

- **self** : RCBF_SAC。
- **memory** : ReplayMemory, 保存真实环境经验。
- **batch_size** : int, 一次抽多少条经验, 默认 256。
- **updates** : int, 到目前为止已经执行过多少次网络更新。
- **dynamics_model** : DynamicsModel。
- **memory_model** : ReplayMemory 或 None, 保存模型虚拟经验。
- **real_ratio** : float 或 None, batch 中真实经验目标比例。

调用位置：[跳到`main.py`的网络更新](#)

4.2 经验中保存了什么

ReplayMemory 每条经验含九项：

```
state
action
reward
next_state
mask
t
next_t
cbf_info
next_cbf_info
```

实现：[跳到`ReplayMemory`](#)

这里的 **state** 实际通常是 policy observation。

4.3 判断是否混合模型经验

```
if memory_model and real_ratio:
```

这是逻辑与。只有两边都被视为真才进入。

- **memory_model** 是 ReplayMemory 对象。它实现了 `__len__`, 空 buffer 可能被视为假, 非空被视为真。
- **real_ratio** 是 float。0.0 被视为假, 非零数被视为真。

因此该分支表示：模型经验池存在、里面可用，并且真实比例不是零。

4.4 从真实经验池采样

```
state_batch, action_batch, reward_batch, next_state_batch, \
mask_batch, t_batch, next_t_batch, cbf_info_batch, \
next_cbf_info_batch = memory.sample(
    batch_size=int(real_ratio * batch_size)
)
```

这是一条很长的 tuple 解包语句。

`memory.sample(...)` 返回九个 NumPy 数组，然后依次赋给左侧九个变量。

假设：

```
batch_size = 256
real_ratio = 0.3

real_ratio * batch_size
```

结果为浮点数 **76.8**。

```
int(76.8)
```

结果为整数 **76**。`int()` 在这里直接去掉小数部分，不进行四舍五入。

典型 shape：

```
state_batch    (76, obs_dim)
action_batch   (76, action_dim)
reward_batch   (76,)
next_state_batch (76, obs_dim)
mask_batch     (76,)
```

4.5 从模型经验池采样

```
state_batch_m, action_batch_m, ... = memory_model.sample(
    batch_size=int((1 - real_ratio) * batch_size)
)
```

后缀 `_m` 表示 model 数据。这只是变量命名约定，没有特殊语法意义。

例子：

```
int((1 - 0.3) * 256)
= int(179.2)
= 179
```

真实 76 条加模型 179 条等于 255 条，不一定严格等于原来的 256。原因是两次 `int()` 都向零截断。

4.6 拼接真实和模型数据

```
state_batch = np.vstack((state_batch, state_batch_m))
```

`np` 是 `numpy` 模块的别名。

`np.vstack(...)` 按行向下拼接二维数组：

```
(76, 7) 和 (179, 7)
-> (255, 7)
```

参数：

```
(state_batch, state_batch_m)
```

是一个包含两个数组的 tuple。

动作和 next state 同样使用 **vstack**：

```
action_batch = np.vstack(...)
next_state_batch = np.vstack(...)
```

reward 和 mask 是一维数组，因此使用：

```
np.hstack(...)
```

结果：

```
(76,) 和 (179,)
-> (255,)
```

CBF 附加信息只有存在时才拼接：

```
if cbf_info_batch is not None and cbf_info_batch[0] is not None:
```

- **is not None**：不是空对象。
- **and**：左右两项必须都为真。
- Python **and** 从左向右执行。如果第一项是假，第二项不会执行，这叫短路求值。

4.7 Model-free 分支

```
else:
    state_batch, action_batch, ... = memory.sample(
        batch_size=batch_size
    )
```

没有模型经验时，全部样本来自真实 replay buffer。

4.8 NumPy 转换成 Tensor

```
state_batch = torch.FloatTensor(state_batch).to(self.device)
```

执行过程：

```
NumPy ndarray
-> 32位浮点 Tensor
-> 移动到 CPU 或 GPU
```

next_state_batch 和 **action_batch** 同理。

```
reward_batch = torch.FloatTensor(reward_batch) \
    .to(self.device) \
    .unsqueeze(1)
```

shape 变化：

```
转换前 reward_batch : (B,)
unsqueeze(1) 后 : (B, 1)
```

为什么需要 **(B, 1)**：critic 每个样本输出一个 Q 值，输出 shape 也是 **(B, 1)**。形状一致才能直接计算 MSE。

mask_batch 同理。

如果存在 CBF 信息，也转换成 Tensor。

4.9 with torch.no_grad() 是什么

```
with torch.no_grad():
```

with 是 Python 上下文管理语句。可以先理解成：

```
进入一个临时区域
在该区域内关闭梯度记录
离开缩进区域后恢复原设置
```

这里计算的是 critic 的监督目标。目标被当作答案，不应通过这条路径反向修改 policy 或 target critic。

4.10 为下一个状态生成动作

```
next_state_action, next_state_log_pi, _ = \
    self.policy.sample(next_state_batch)
```

输入：

```
next_state_batch : Tensor, shape (B, obs_dim)
```

输出：

```
next_state_action : Tensor, shape (B, action_dim)
next_state_log_pi : Tensor, shape (B, 1)
第三项均值动作被 '_' 忽略
```

4.11 full 和 mod 的安全动作

```
if self.cbf_mode == 'full' or self.cbf_mode == 'mod':
```

or 是逻辑或，任意一项为真就进入。

```
modular=self.cbf_mode == 'mod'
```

先计算右侧比较：

```
模式为 mod -> True
其他模式 -> False
```

然后把这个 bool 作为 **modular** 参数传入。

```
next_state_action = self.get_safe_action(...)
```

原来的 Tensor 变量名被重新赋值，现在引用安全动作 Tensor。

4.12 Target Critic 计算下一个状态价值

```
qf1_next_target, qf2_next_target = \
    self.critic_target(next_state_batch, next_state_action)
```

self.critic_target(...) 看起来像函数调用，因为 PyTorch 神经网络对象实现了 **__call__**。它最终调用网络的 **forward()**。

输出：

```
qf1_next_target : Tensor, shape (B, 1)
qf2_next_target : Tensor, shape (B, 1)

torch.min(qf1_next_target, qf2_next_target)
```

这是对应位置逐元素取较小值，不是把整个数组压缩成一个最小数字。

```
min_qf_next_target = torch.min(...) \
    - self.alpha * next_state_log_pi
```

每个样本都计算：

```
min(Q1_target, Q2_target) - alpha * log_pi
```

shape 仍为 **(B, 1)**。

4.13 Bellman 目标

```
next_q_value = reward_batch + \
    mask_batch * self.gamma * min_qf_next_target
```

每条经验的公式：

```
目标Q =
    当前 reward
    + mask × gamma ×
    (较小的 target Q - alpha × log_pi)
```

如果任务真正结束，**mask=0**：

```
目标Q = reward
```

如果没有真正结束，**mask=1**，才考虑未来价值。

输出：

```
next_q_value : Tensor, shape (B, 1)
```

4.14 普通 Critic 当前预测

```
qf1, qf2 = self.critic(state_batch, action_batch)
```

输入：

```
state_batch Tensor (B, obs_dim)
action_batch Tensor (B, action_dim)
```

输出：

```
qf1 Tensor (B, 1)
qf2 Tensor (B, 1)
```

注意：这里不在 **torch.no_grad()** 里面，所以 PyTorch 会记录计算图，稍后可以对 critic 求梯度。

4.15 两个 Critic 的 MSE

```
qf1_loss = F.mse_loss(qf1, next_q_value)
qf2_loss = F.mse_loss(qf2, next_q_value)
```

F 是：

```
torch.nn.functional
```

的别名。

.mse_loss 是模块中的函数。MSE 表示均方误差：

```
先计算每个样本的 (预测Q - 目标Q)^2
再对 batch 求平均
```

两者输出都是只含一个数字的 Tensor，shape 通常为 **()**，称为标量 Tensor。

```
qf_loss = qf1_loss + qf2_loss
```

把两个 critic 的误差相加，仍是标量 Tensor。

4.16 更新 Critic 的固定三步

```
self.critic_optim.zero_grad()
```

清除 critic 参数上一次更新留下的梯度。PyTorch 默认会累加梯度，所以不能省略。

```
qf_loss.backward()
```

从 loss 反向计算每个 critic 参数的梯度。

```
self.critic_optim.step()
```

Adam 根据梯度真正修改 critic 参数。

记忆模板：

```
zero_grad 清旧梯度  
backward 算新梯度  
step 改参数
```

4.17 为 Policy 更新重新采样动作

```
pi, log_pi, _ = self.policy.sample(state_batch)
```

- **pi**：当前状态下 policy 的随机动作 Tensor。
- **log_pi**：动作对数概率 Tensor。
- **_**：忽略均值动作。

这里不使用刚才 replay buffer 中的 **action_batch**，因为现在要问的是：

当前最新版 policy 在这些状态下会选择什么动作？

4.18 Policy 更新是否经过可微安全层

```
if self.cbf_mode == 'full' or self.cbf_mode == 'mod':  
    pi = self.get_safe_action(...)
```

full 模式下，梯度路径是：

```
policy 参数  
-> policy 原始动作  
-> 可微 CBF-QP  
-> 安全动作  
-> critic  
-> policy loss
```

baseline 不在这里调用安全层，所以训练 loss 不通过 QP 反向传播。

4.19 Critic 评价当前 Policy 动作

```
qf1_pi, qf2_pi = self.critic(state_batch, pi)  
min_qf_pi = torch.min(qf1_pi, qf2_pi)
```

这里没有更新 critic。只是利用 critic 给 policy 动作打分。

输出 shape 都是 **(B, 1)**。

4.20 Policy Loss

```
policy_loss = (  
    (self.alpha * log_pi) - min_qf_pi  
).mean()
```

括号中的每个样本都有一个 loss：

$$\alpha \times \log_{\pi} - \min(Q1, Q2)$$

.mean() 是 Tensor 成员函数，对所有样本求平均，得到标量 Tensor。

优化器要最小化 loss：

- **-Q** 越小意味着 Q 越大，因此鼓励高价值动作。
- **alpha * log_pi** 是 SAC 的熵相关项，帮助保留探索。

随后仍是固定三步：

```
self.policy_optim.zero_grad()  
policy_loss.backward()  
self.policy_optim.step()
```

这次修改的是 policy 参数。

4.21 自动更新 Alpha

```
if self.automatic_entropy_tuning:
```

启用时计算：

```
alpha_loss = -(  
    self.log_alpha *  
    (log_pi + self.target_entropy).detach()  
).mean()
```

先看：

```
(log_pi + self.target_entropy).detach()
```

.detach() 表示更新 alpha 时，把这部分当作普通目标数字，不利用 alpha loss 去修改 policy。

再用三步更新：

```
self.alpha_optim.zero_grad()  
alpha_loss.backward()  
self.alpha_optim.step()
```

只修改 **self.log_alpha**。

```
self.alpha = self.log_alpha.exp()
```

.exp() 对 Tensor 每个元素求自然指数。

```
log_alpha = 0 -> alpha = exp(0) = 1
```

使用指数可以保证 alpha 始终为正数。

```
alpha_tlogs = self.alpha.clone()
```

.clone() 创建内容相同的新 Tensor，供日志使用。

如果关闭自动调节：

```
alpha_loss = torch.tensor(0.).to(self.device)
```

```
alpha_tlogs = torch.tensor(self.alpha)
```

人为创建零 loss 和当前 alpha 的 Tensor，保证函数两条分支最后都能返回相同数量的结果。

4.22 Target Critic 软更新

```
if updates % self.target_update_interval == 0:
```

% 是取余运算，和 C/C++ 一样。

默认 interval 为 1，所以每次：

```
updates % 1 == 0
```

都会成立。

```
soft_update(self.critic_target, self.critic, self.tau)
```

实现：[跳到 'soft_update'](#)

每个参数执行：

```
target新值 =  
target旧值 × (1 - tau)  
+ critic当前值 × tau
```

默认 **tau=0.005**：

```
target新值 =  
99.5% 的旧 target  
+ 0.5% 的当前 critic
```

4.23 返回日志数字

```
return (  
    qf1_loss.item(),  
    qf2_loss.item(),  
    policy_loss.item(),  
    alpha_loss.item(),  
    alpha_tlogs.item()  
)
```

.item() 把单元素 Tensor 转成普通 Python 数字。

函数返回一个五元素 tuple：

```
(float, float, float, float, float)
```

调用者使用 tuple 解包：

```
critic_1_loss, critic_2_loss, policy_loss, ent_loss, alpha = \  
agent.update_parameters(...)
```

这些普通数字主要用于打印和 Comet 日志，不再用于反向传播。

5. update_parameters_compensator

源码入口：[跳到该函数](#)

```
def update_parameters_compensator(self, comp_rollouts):
```

参数：

- **self**：RCBF_SAC。

- `comp_rollouts` : `list`, 列表中的每个元素是一个 episode 的 `dict`。

每个字典大致包含：

```
obs 多步 observation
u_safe 多步 CBF 修正
u_comp 多步 compensator 输出
```

```
if self.compensator:
```

只有创建了 `compensator` 对象才执行。

```
self.compensator.train(comp_rollouts)
```

调用 `Compensator` 对象的 `train` 成员函数。

[跳到 `Compensator.train`](#)

该函数本身不返回结果，Python 默认返回 `None`。

6. `save_model` : 保存网络参数

源码入口：[跳到 `save\_model`](#)

6.1 参数

```
def save_model(self, output):
```

- `self` : `RCBF_SAC`。
- `output` : `str`, 输出目录路径，例如 `output/Unicycle-run1`。

6.2 打印保存位置

```
print('Saving models in {}'.format(output))
```

`.format(output)` 是字符串成员函数调用。

```
'Saving models in {}'.format('output/Unicycle-run1')
```

得到：

```
Saving models in output/Unicycle-run1
```

花括号 `{}` 是待替换位置。

6.3 保存 Actor

```
torch.save(
    self.policy.state_dict(),
    '{}actor.pkl'.format(output)
)
```

```
self.policy.state_dict()
```

返回一个类似字典的对象，里面保存每层参数名称和 `Tensor`，例如权重、偏置。

它保存的是参数，不是完整 Python `policy` 对象。

路径：

```
output/Unicycle-run1/actor.pkl
```

6.4 保存 Critic

```
torch.save(
    self.critic.state_dict(),
    '{}critic.pkl'.format(output)
)
```

`self.critic` 内部同时包含 Q1 和 Q2，所以一个 `critic.pkl` 会保存两套 critic 参数。

6.5 可选保存 Compensator

```
if self.compensator:
    self.compensator.save_model(output)
```

存在 compensator 时会额外创建：

```
comp_actor.pkl
```

6.6 当前没有保存的内容

当前函数没有保存：

- `critic_target`
- critic 和 policy 的 Adam 优化器状态
- `log_alpha`
- `alpha_optim`
- 当前 episode 和 update 计数

所以它适合测试和基础加载，但不是严格意义上可以完全无损续训的 checkpoint。

7. `load_weights`：加载网络参数

源码入口：[跳到 `load\_weights`](#)

7.1 参数和提前返回

```
def load_weights(self, output):
```

- `output`：str 或 None。

```
    if output is None: return
```

这是把两条语句压在同一行：

```
    if output is None:
        return
```

`return` 后面没有值，所以返回 `None`，函数立即结束。

7.2 加载 Actor

```
self.policy.load_state_dict(
    torch.load(
        '{}actor.pkl'.format(output),
        map_location=self.device
    )
)
```

从内向外：

1. `format` 得到 actor 文件路径。
2. `torch.load` 从磁盘读取参数字典。
3. `map_location=self.device` 要求把读取的 Tensor 映射到当前 CPU 或 GPU。
4. `load_state_dict(...)` 把参数写入已经创建好的 policy 网络。

注意：程序会先在 `__init__` 中按照当前配置创建网络，再把磁盘参数装进去。

7.3 加载 Critic

```
self.critic.load_state_dict(
    torch.load(
        '{}critic.pkl'.format(output),
        map_location=self.device
    )
)
```

流程与 actor 相同。

7.4 加载可选 Compensator

```
if self.compensator:
    self.compensator.load_weights(output)
```

只有当前启动参数要求创建 compensator 时才加载它。

7.5 一个重要的续训限制

加载后只更新了：

```
self.policy
self.critic
可选 self.compensator
```

没有执行：

```
hard_update(self.critic_target, self.critic)
```

因此刚加载后，`critic_target` 仍然是本次程序启动时随机初始化并复制的旧参数，不是刚从文件读取的 critic 参数。

测试过程主要使用 policy，通常影响较小；严格续训时，这是需要单独实现的实现限制。

8. `get_safe_action`：调用动力学模型和 CBF-QP

源码入口：[跳到 `get\_safe\_action`](#)

8.1 参数和返回类型

```
def get_safe_action(
    self,
    obs_batch,
    action_batch,
    dynamics_model,
    modular=False,
    cbf_info_batch=None
):
```

- `self` : `RCBF_SAC`。
- `obs_batch` : `torch.Tensor`, shape 通常是 `(B, obs_dim)`。
- `action_batch` : `torch.Tensor`, shape 通常是 `(B, action_dim)`。
- `dynamics_model` : `DynamicsModel`。
- `modular` : `bool`。
- `cbf_info_batch` : `torch.Tensor` 或 `None`。

返回：

```
safe_action_batch : torch.Tensor
shape 与 action_batch 相同
```

8.2 Observation 转物理 State

```
state_batch = dynamics_model.get_state(obs_batch)
```

调用 `DynamicsModel` 的成员函数：

[跳到 `'DynamicsModel.get\_state'`](#)

Unicycle 中：

```
obs_batch Tensor, shape (B, 7)
state_batch Tensor, shape (B, 3)
```

物理 state 是：

```
[x, y, theta]
```

Policy observation 和动力学 state 不是同一个概念。

8.3 预测扰动

```
mean_pred_batch, sigma_pred_batch = \
    dynamics_model.predict_disturbance(state_batch)
```

函数返回一个二元素 tuple，随后解包。

- `mean_pred_batch` : 预测扰动均值，Tensor。
- `sigma_pred_batch` : 预测扰动标准差，Tensor。
- 两者 shape 通常与 `state_batch` 相同。

实现：[跳到 `'predict\_disturbance'`](#)

如果 GP 尚未训练，代码使用零均值和预设最大标准差。

8.4 调用 CBF-QP 层

```
safe_action_batch = self.cbf_layer.get_safe_action(
    state_batch,
    action_batch,
    mean_pred_batch,
    sigma_pred_batch,
    modular=modular,
    cbf_info_batch=cbf_info_batch
)
```

`self.cbf_layer` 是 `CBFQPLayer` 对象。

CBF 层入口：[跳到 `CBFQPLayer.get_safe_action``](#)

它接收：

- 物理状态
- policy 名义动作
- 扰动均值
- 扰动不确定性
- 是否 modular
- 可选额外安全信息

普通 full/baseline 模式下，内部计算：

最终动作 = 名义动作 + QP求得的最小安全修正

`modular=True` 且环境不是 PVTOL 时，当前代码主要把动作裁剪到允许范围，不加入完整障碍物 CBF 约束。

8.5 返回

```
return safe_action_batch
```

这里不转换成 NumPy，也不调用 `.detach()`。

原因是这个函数既可能在环境交互时调用，也可能在 `full` 模式的 policy loss 内调用。训练时必须保留 Tensor 和计算图，梯度才能穿过可微 QP 回到 policy。

9. 七个函数怎样连起来

9.1 程序启动

```
main.py
-> 创建 env
-> RCBF_SAC.__init__
-> 创建 critic、target critic、policy
-> 可选创建 CBF layer 和 compensator
```

9.2 与真实环境交互

```
main.py 当前 observation
-> select_action
-> policy 或随机动作
-> 可选 compensator
-> get_safe_action
-> DynamicsModel 提取物理 state
-> CBFQPLayer 求安全动作
-> 转 NumPy
-> env.step(action)
```


9.3 网络更新

main.py
-> update_parameters
-> ReplayMemory.sample
-> 计算 target Q
-> 更新 critic
-> 计算 policy loss
-> full/mod 时调用 get_safe_action
-> 更新 policy
-> 更新 alpha
-> soft_update target critic

9.4 保存和测试

训练中每隔若干 episode
-> save_model
-> actor.pkl、critic.pkl

测试启动
-> load_weights
-> select_action(evaluate=True)
-> 使用均值动作

10. 四种 CBF 模式在这些函数中的位置

off

__init__ 不创建 CBFQPlayer
select_action 由 main.py 传 safe_action=False
update_parameters 不把 policy 动作送入安全层

baseline

select_action 与环境交互时使用完整安全动作
update_parameters 的 critic target 和 policy loss 不经过 QP
replay buffer 保存修正前的 policy 动作

full

与环境交互时使用完整安全动作
critic target 使用安全动作
policy loss 使用可微 QP 后的安全动作

mod

与真实环境交互时仍使用完整安全层
网络更新时调用 modular=True
Unicycle/SimulatedCars 中训练安全动作主要退化为动作裁剪

11. 看到这些写法时应该立刻翻译

x.y

翻译：读取 **x** 对象里的成员 **y**。

x.y(...)

翻译：调用 **x** 对象的成员函数 **y**。

x(...)

翻译：调用函数，或者调用实现了 `__call__` 的对象。

```
x.shape[0]
```

翻译：先读取 shape 元组，再取第 0 个维度大小。

```
a, b = function()
```

翻译：函数返回一个至少含两个元素的结构，依次拆给 **a** 和 **b**。

```
-
```

翻译：这个返回值不打算使用。

```
x is None
```

翻译：检查 **x** 是否是空对象 **None**。

```
if x:
```

翻译：让 Python 对 **x** 做隐式真假判断。遇到 NumPy 数组和 Tensor 时需要格外小心。

```
x.to(device)
```

翻译：把 Tensor 或神经网络移动到 CPU/GPU，返回的仍是对应的 Tensor/网络对象。

```
x.unsqueeze(1)
```

翻译：在第 1 个位置增加长度为 1 的维度。

```
x.detach().cpu().numpy()
```

翻译：断开梯度，移动到 CPU，转换成 NumPy 数组。

```
loss.backward()
```

翻译：从 loss 反向计算梯度，不会自动修改参数。

```
optimizer.step()
```

翻译：优化器根据已有梯度真正修改参数。

12. 最后应该掌握的对象关系

```
RCBF_SAC agent
|
|-- policy      根据 observation 产生动作
|-- policy_optim 修改 policy 参数
|
|-- critic      Q1、Q2，评价 state-action
|-- critic_optim 修改 critic 参数
|
|-- critic_target  critic 的慢速副本
|
|-- log_alpha    可学习的探索温度参数
|-- alpha_optim  修改 log_alpha
|
|-- cbf_layer     把名义动作修正为安全动作
|
|-- compensator  可选，学习近似安全修正
|
|-- env          当前环境对象
|-- action_space 动作上下界和维数
|-- device       CPU 或 GPU
```

读代码时先问“它是谁的对象”，再问“它在算法里做什么”。如果反过来直接从公式猜代码，Python 的动态类型、PyTorch 的 Tensor 和强化学习概念会同时压过来，认知负担会非常大。